

Практическая работа 1

Введение в ИИ для кибербезопасности и настройка среды

1. Цели и задачи занятия

Цель занятия: Сформировать у студентов практические навыки по развертыванию лабораторной среды для проведения экспериментов по применению искусственного интеллекта в задачах обнаружения вторжений, а также получить первичное представление о роли ИИ в кибербезопасности.

Основные задачи:

1. Изучить теоретические основы применения ИИ в задачах кибербезопасности;
2. Установить и настроить программное обеспечение для проведения исследований;
3. Познакомиться с базовыми библиотеками машинного обучения;
4. Выполнить первичный анализ данных для задач обнаружения вторжений.

2. Теоретическая часть: ИИ в кибербезопасности

2.1 Роль искусственного интеллекта в обнаружении вторжений

Современные системы обнаружения вторжений (IDS) сталкиваются с рядом вызовов в области информационной безопасности, к которым можно отнести: растущий объем сетевого трафика, появление новых типов атак, необходимость обработки данных в реальном времени. Искусственный интеллект предлагает эффективные решения этих проблем.

Основные направления применения ИИ в кибербезопасности:

Обнаружение угроз (Threat Detection): выявление вредоносного ПО, фишинговых атак и сетевых аномалий с помощью моделей машинного обучения

Анализ вредоносного ПО (Malware Analysis): автоматическая классификация вредоносных образцов и поведенческий анализ

Автоматизация SOC (SOC Automation): интеллектуальная обработка инцидентов и приоритезация оповещений

Threat Intelligence: сбор и анализ данных об угрозах из различных источников

2.2 Типы машинного обучения в задачах IDS

Для обнаружения вторжений применяются различные подходы машинного обучения :

Тип обучения	Описание	Применение в IDS
Обучение с учителем	Модели обучаются на размеченных данных (нормальный трафик / атаки)	Классификация известных типов атак
Обучение без учителя	Поиск закономерностей в неразмеченных данных	Обнаружение новых, неизвестных ранее атак
Обучение с подкреплением	Агент учится на опыте взаимодействия со средой	Адаптивные системы защиты

2.3 Обзор инструментария

Для практической работы потребуется следующее программное обеспечение :

Базовое ПО:

Python 3.x — основной язык программирования

Jupyter Notebook — интерактивная среда для анализа данных и прототипирования моделей

Библиотеки машинного обучения:

Scikit-learn – классические алгоритмы ML;

Tensorflow / keras – глубокое обучение;

Pandas / numpy – обработка данных;

matplotlib / seaborn – визуализация.

3. Практическая часть: Настройка среды

3.1 Установка Python и Jupyter Notebook

Вариант 1: Установка через Anaconda (рекомендуется для начинающих)

Anaconda — дистрибутив Python, включающий большинство необходимых библиотек и менеджер пакетов .

Шаги установки:

1. Перейдите на официальный сайт Anaconda
2. Скачайте установщик для вашей операционной системы
3. Запустите установщик и следуйте инструкциям
4. После установки запустите **Anaconda Navigator**
5. В интерфейсе Navigator нажмите **Launch** под Jupyter Notebook

Вариант 2: Установка через pip (легковесный вариант)

Если Python уже установлен, можно установить Jupyter Notebook с помощью менеджера пакетов pip :

```
bash
# Установка Jupyter Notebook
pip install notebook
# Запуск Jupyter Notebook
jupyter notebook
```

Для Linux (Ubuntu/Debian):

```
bash
# Обновление репозиториев
sudo apt update
# Установка pip и Python (если не установлены)
sudo apt install python3-pip
# Установка Jupyter Notebook
pip3 install notebook
# Запуск
jupyter notebook
```

Для macOS:

```
bash
# Установка через pip
pip3 install notebook

# Запуск
jupyter notebook
```

3.2 Установка библиотек машинного обучения

После установки Jupyter Notebook необходимо установить библиотеки для работы с ИИ :

```
bash
# Установка основных библиотек
pip install scikit-learn tensorflow keras pandas numpy matplotlib
jupyter

# Для работы с форматом HDF5 (может потребоваться для некоторых д
атасетов)
pip install h5py
```

Проверка установки:

Создайте новый Jupyter Notebook и выполните следующие команды:

```
python
# Проверка импорта библиотек
import sklearn
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
print("Scikit-learn version:", sklearn.__version__)
print("TensorFlow version:", tf.__version__)
print("Pandas version:", pd.__version__)
print("NumPy version:", np.__version__)
print("Все библиотеки успешно загружены!")
```

3.3 Создание виртуального окружения (опционально, но рекомендуется)

Для изоляции проекта рекомендуется создать виртуальное окружение :

```
bash
# Создание виртуального окружения с Python 3.11
conda create -y -n ids-ai python=3.11 ipykernel

# Активация окружения
conda activate ids-ai

# Установка необходимых пакетов
pip install scikit-learn tensorflow keras pandas numpy matplotlib
jupyter h5py
```

Регистрация ядра для Jupyter

```
python -m ipykernel install --name ids-ai --user
```

3.4 Знакомство с датасетами для обнаружения вторжений

Для тренировки моделей обнаружения вторжений используются специализированные датасеты :

Датасет	Описание	Тип данных	Размер
CICIDS2017	Современный датасет с различными типами атак	Сетевой трафик (PCAP)	~50 ГБ
NSL-KDD	Улучшенная версия KDD Cup 99	Сетевые соединения	~4 МБ
EMBER Malware Dataset	Для обнаружения вредоносного ПО	PE-файлы	~5 ГБ
SpamAssassin	Фишинговые и легитимные email-сообщения	Текстовые данные	~10 МБ

Для первого знакомства рекомендуется использовать NSL-KDD как небольшой и хорошо изученный датасет.

Загрузка примера данных:

```
python
```

```
import pandas as pd
```

```
import urllib.request
```

```
import os
```

```
# Создание директории для данных
```

```
os.makedirs("data", exist_ok=True)
```

```
# Загрузка небольшого примера (KDDTrain+)
```

```
url = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain%2B.csv"
```

```
local_path = "data/KDDTrain.csv"
```

```

print("Загрузка данных...")
urllib.request.urlretrieve(url, local_path)
print(f"Данные сохранены в {local_path}")

# Загрузка в pandas
column_names = [
    'duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',
    'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in',
    'num_compromised', 'root_shell', 'su_attempted', 'num_root', 'num_file_creations',
    'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login',
    'is_guest_login', 'count', 'srv_count', 'error_rate', 'srv_error_rate',
    'error_rate', 'srv_error_rate', 'same_srv_rate', 'diff_srv_rate',
    'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
    'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
    'dst_host_srv_diff_host_rate', 'dst_host_error_rate', 'dst_host_srv_error_rate',
    'dst_host_error_rate', 'dst_host_srv_error_rate', 'label', 'difficulty'
]

data = pd.read_csv(local_path, names=column_names)
print(f"Загружено записей: {len(data)}")
print("\nПервые 5 строк данных:")
print(data.head())

print("\nИнформация о датасете:")
print(f"Количество признаков: {data.shape[1] - 2}") # минус label и difficulty
print(f"Типы атак: {data['label'].unique()}")
print(f"\nРаспределение классов:")
print(data['label'].value_counts())

```

3.5 Базовый анализ данных

Выполним первичный разведочный анализ данных (EDA) :

```

python
import matplotlib.pyplot as plt
import seaborn as sns

# Настройка стиля графиков
plt.style.use('ggplot')
sns.set(style="whitegrid")

# 1. Распределение классов
plt.figure(figsize=(12, 5))

```

```

plt.subplot(1, 2, 1)
label_counts = data['label'].value_counts()
plt.bar(label_counts.index, label_counts.values)
plt.title('Распределение классов атак')
plt.xlabel('Тип')
plt.ylabel('Количество')
plt.xticks(rotation=45)

# 2. Анализ протоколов
plt.subplot(1, 2, 2)
protocol_counts = data['protocol_type'].value_counts()
plt.pie(protocol_counts.values, labels=protocol_counts.index, autopct='%1.1f%%')
plt.title('Распределение протоколов')

plt.tight_layout()
plt.show()

# 3. Статистика числовых признаков
numeric_features = ['duration', 'src_bytes', 'dst_bytes', 'count']
print("\nСтатистика основных числовых признаков:")
print(data[numeric_features].describe())

# 4. Проверка на пропущенные значения
print("\nПропущенные значения в данных:")
print(data.isnull().sum().sum())

```

3.6 Настройка среды для работы с AI-инструментами в Kali Linux (для продвинутых)

Если вы используете Kali Linux для исследований в области безопасности, можно установить дополнительные AI-инструменты :

```

bash
# Установка Python-библиотек для машинного обучения
sudo apt install python3-sklearn python3-tensorflow

# Установка специализированных инструментов
# DeepExploit - AI-управляемый пентестинг
git clone https://github.com/13o-bbr-bbq/machine_learning_security.git
cd machine_learning_security/DeepExploit

```

```
pip install -r requirements.txt
```

SnortAI - AI-усиленная IDS

```
git clone https://github.com/lorcoom7/SnortAI.git
```

```
cd SnortAI
```

```
pip install -e .
```

4. Практические задания для самостоятельного выполнения

Задание 1: Установка и настройка среды (обязательное)

1. Установите Python и Jupyter Notebook одним из предложенных способов
2. Установите библиотеки: scikit-learn, tensorflow, keras, pandas, numpy, matplotlib
3. Создайте новый Jupyter Notebook и проверьте импорт всех библиотек
4. Сохраните скриншот успешного выполнения

Задание 2: Загрузка и первичный анализ датасета (обязательное)

1. Загрузите датасет NSL-KDD по приведенной выше ссылке
2. Выполните первичный анализ:
 - Определите количество образцов каждого класса атак
 - Постройте график распределения классов
 - Выявите дисбаланс классов
3. Определите, какие признаки являются категориальными, а какие числовыми

Задание 3: Работа с виртуальным окружением (дополнительное)

1. Создайте виртуальное окружение conda или venv для проекта
2. Установите необходимые пакеты в изолированном окружении
3. Настройте Jupyter для работы с созданным ядром

Задание 4: Исследование AI-инструментов (для продвинутых)

1. Ознакомьтесь с инструментом DeepExploit или SnortAI
2. Установите инструмент в тестовой среде (лучше использовать виртуальную машину)
3. Изучите документацию и подготовьте краткий отчет о возможностях инструмента

5. Контрольные вопросы

1. Какие задачи кибербезопасности могут быть решены с помощью искусственного интеллекта?
2. В чем разница между обучением с учителем и без учителя применительно к обнаружению вторжений?
3. Почему для задач обнаружения вторжений важно использовать специализированные датасеты, такие как CICIDS2017 или NSL-KDD?
4. Какие библиотеки машинного обучения наиболее популярны для задач кибербезопасности и почему?
5. Что такое дисбаланс классов и почему он представляет проблему для систем обнаружения вторжений?

6. Дополнительные ресурсы

Онлайн-платформы для практики

- **Google Colab** — бесплатная облачная среда с GPU для тренировки моделей
- **Kaggle** — платформа с датасетами и соревнованиями по ML
- **TryHackMe** — комнаты по AI в кибербезопасности
- **Hack The Box** — реальные пентестинг-лаборатории

Литература

- Калинин М.О., Крундышев В.М. Технологии машинного обучения в кибербезопасности. Основы искусственного интеллекта. Лабораторный практикум. — СПб.: ПОЛИТЕХ-ПРЕСС, 2024

7. Требования к отчету

Отчет по практическому занятию должен содержать:

1. Титульный лист с указанием темы занятия
2. Краткое теоретическое введение (1-2 страницы)
3. Описание выполненных действий с указанием команд и их результатов
4. Скриншоты успешной установки и работы ПО
5. Результаты анализа датасета (графики, статистики)
6. Ответы на контрольные вопросы
7. Выводы по проделанной работе